

Introduction to Deep Learning

Assignment 2

Coen Büller (s2631415), Mario Magalini (s4949404)

December 8, 2025

Introduction

This report explores Generative Models (VAE, GAN, and CAE) trained on fractal image datasets and investigates an encoder-decoder recurrent network for sequence modeling applied to simple arithmetic.

1 Generative Models

Generative models are a family of deep unsupervised learning methods that aim to capture the underlying distribution of a dataset and use it to generate new samples. In this section, two such architectures are explored: the Variational Autoencoder (VAE) and the Generative Adversarial Network (GAN). A Convolutional Autoencoder (CAE) is also included as a baseline, following the structure provided in the assignment notebook, since its encoder and decoder blocks form the foundation from which the VAE and GAN architectures are built. All models are trained on fractal-image datasets, and their learned latent spaces are used to produce novel samples, including linear interpolations between latent points.

1.1 Working principles of the generative models

The three models used in this work share the same general structure: a convolutional encoder that compresses the input image into a latent representation, and a deconvolutional decoder that reconstructs it. All architectures consist of four convolutional layers in the encoder, progressively reducing the spatial resolution from 64×64 to 32×32 , 16×16 , 8×8 , and finally 4×4 . The resulting feature map is flattened and mapped to a latent vector of 512 components. The decoder mirrors this structure with four deconvolutional layers that upsample the latent representation back to the original image size. ReLU activations are used in all intermediate layers, while a sigmoid activation is applied at the final decoder layer to map pixel values to $[0, 1]$. The reconstruction loss is the binary cross-entropy, treating each pixel as an independent. This compression-decompression process inevitably discards fine details, producing smoother and blurrier reconstructions.

The Variational Autoencoder (VAE) extends the CAE by introducing a probabilistic latent space. The encoder still relies on the same convolutional backbone but, instead of producing a single latent vector, outputs two 128-dimensional vectors: z_{mean} and z_{var} , representing the mean and log-variance of a Gaussian distribution over latent variables. A custom sampling layer implements the reparameterisation trick:

$$z = z_{\text{mean}} + \exp(0.5 z_{\text{var}}) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

The decoder is identical to the CAE decoder. Training minimises the sum of the reconstruction loss and the KL divergence between $q(z|x)$ and the standard normal prior, corresponding to the Evidence Lower Bound (ELBO).

The GAN follows a different training paradigm, although it also relies on convolutional and deconvolutional blocks. A 256-dimensional noise vector is fed into the generator, which maps it to a synthetic image. The final layer of the generator uses a tanh activation, since the training images are scaled to the $[-1, 1]$ range. Generated images are mixed with real samples and passed to the discriminator, a convolutional classifier trained to assign label 1 to real images and 0 to generated ones. During generator updates, the discriminator is frozen (`trainable = False`) so that only the generator's parameters are modified. Both networks use binary cross-entropy losses. This adversarial setup encourages the generator to produce images that the discriminator cannot distinguish from real ones, typically yielding sharper outputs than the VAE.

1.2 Methodology

The dataset used in this work was created from an adapted version of the Fractals Dream Dataset notebook available on Kaggle [1]. The original pipeline generated 6027 RGB images at resolution $128 \times 128 \times 3$, covering several strange attractors and three colour palettes. This collection was too small and heterogeneous to train generative models reliably, so the pipeline was extended to produce larger and more controlled datasets at resolution $64 \times 64 \times 3$. All datasets were saved in NumPy `.npy` format, published on HuggingFace [2], normalised to $[0, 1]$, and constrained to contain at least 15% non-zero pixels.

Three datasets were produced. (i) A multi-attractor dataset of 24 000 labelled images (`fractal_images.npy`, `fractal_labels.npy`), containing four attractor families (Fractal Dream, Hopalong, De Jong, Clifford) with 6000 images per class, uniformly distributed across the inferno, viridis, and fire palettes. (ii) A Hopalong-only dataset of 18 000 images (`hopalong_images.npy`, `hopalong_labels.npy`), labelled according to palette (0–2). (iii) A Dream-only dataset of 18 000 images (`dream_images.npy`, `dream_labels.npy`), also labelled by palette (0–2).

These datasets allow us to study how structural variability and colour diversity affect the behaviour of generative models.

1.3 Experiments

The CAE, VAE, and GAN models from the assignment notebook were used as a starting point and modified to improve training stability and model capacity. For the VAE, the number of convolutional filters was increased from 64 to 128, the latent dimension from 64 to 128, and the training duration from 20 to 50 epochs. For the GAN, the Adam optimiser was made more stable by reducing the learning rate from 0.0002 to 0.0001 and lowering β_1 from 0.5 to 0.25, while the CAE was left unchanged and used only as a reconstruction baseline. A complete overview of the baseline architectures is provided in Appendix 6.

All models were trained on three representative datasets: the 24 000-image multi-attractor dataset (maximum variability), the 18 000-image Hopalong-only dataset using all colour palettes, and the 18 000-image Dream-only dataset. For the Dream dataset, however, only the inferno palette was used, resulting in a reduced training set of 6000 images.

For each dataset, a fixed set of generated samples was produced for visual comparison with real images. Latent-space interpolations were also computed for both the VAE and the GAN.

For the VAE, two interpolation types were considered: one using latent vectors obtained by encoding two real images, and one using endpoints sampled from a standard normal distribution. In both cases, seven intermediate latent points were generated and decoded. For the GAN, two random latent vectors were sampled, seven intermediate points were computed, and all nine vectors were passed through the generator to assess the continuity of the learned latent manifold.

1.4 Results

In this section we present the qualitative results obtained from the CAE, VAE, and GAN models trained on three representative datasets. A selection of generated samples is shown for each model and dataset, covering the full range of structural and colour variability present in the data. This allows us to assess how each architecture behaves under increasing levels of complexity.

Dataset overview Figure 1 shows one representative grid from each dataset: the multi-attractor dataset, the Hopalong-only dataset with mixed colour palettes, and the Dream-only dataset restricted to the inferno palette. These three cases span different combinations of structural and colour variability: the multi-attractor dataset combines high variability in both geometry and colour palettes; the Hopalong-only dataset has a relatively stable global structure but varying colour palettes; the Dream-only dataset, instead, exhibits high structural variability under a fixed colour palette.

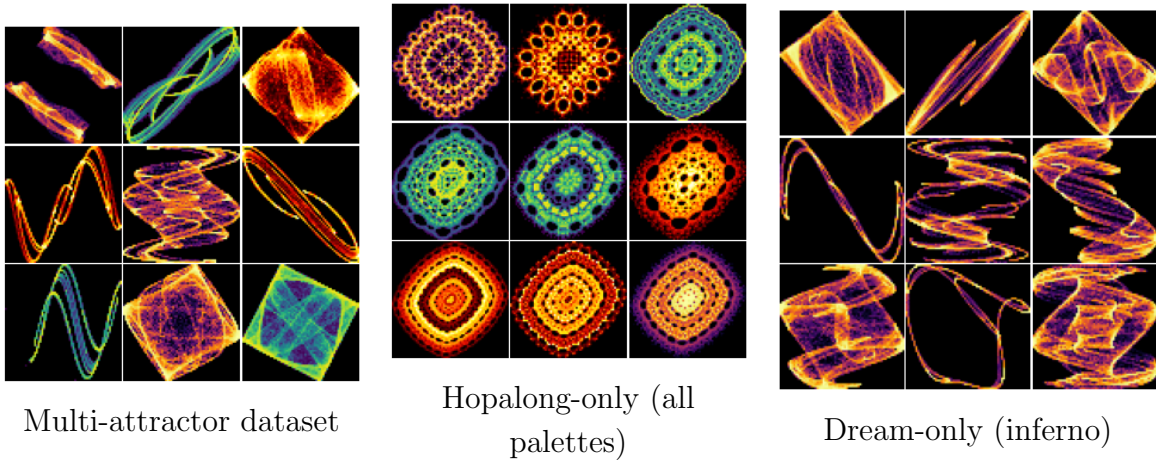


Figure 1: Representative samples from the three datasets used for visual evaluation.

CAE reconstructions The CAE is shown only for the multi-attractor dataset, as its behaviour does not significantly change across datasets. In all cases, the model reconstructs the global shape of the input but consistently loses fine details, producing smooth and slightly blurred outputs.

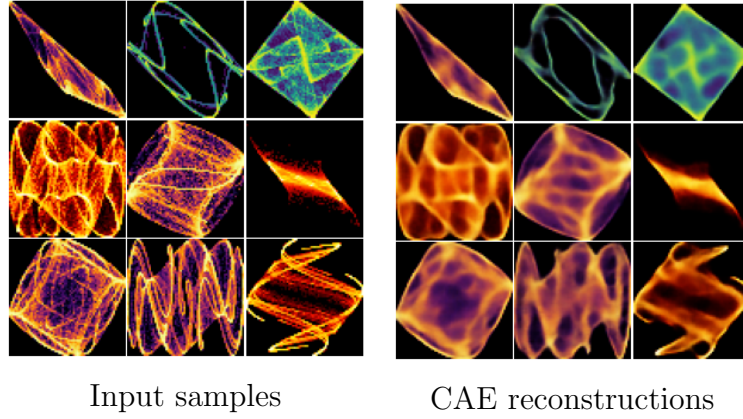


Figure 2: CAE reconstructions on the multi-attractor dataset.

VAE results The behaviour of the VAE reflects the different forms of variability present in the three datasets. On the multi-attractor dataset—characterised by both high structural variability and mixed colour palettes—the model produces smooth, low-contrast samples that often lack clear geometric organisation. A similar loss of structure is observed on the Dream-only dataset, where the palette is fixed but the underlying geometry varies significantly, making it difficult for the VAE to capture consistent bright features. In contrast, the Hopalong dataset presents a stable global structure, and the generated images become more coherent, although still blurrier than those produced by the GAN. When multiple colour palettes are present (multi-attractor and Hopalong), the VAE also mixes colours inconsistently, producing local patches from different palettes.

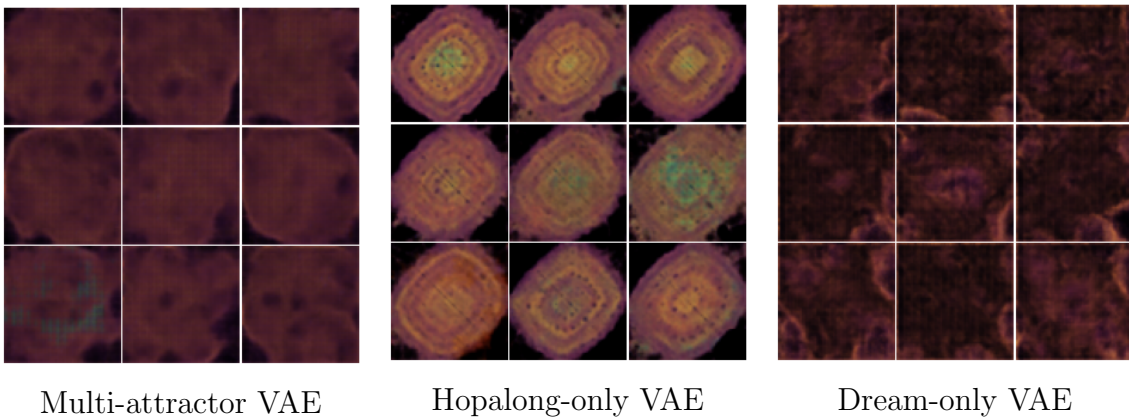


Figure 3: VAE-generated samples for the three datasets.

GAN results The GAN produces sharper samples than the VAE but still varies across datasets. On the multi-attractor dataset, the model fails to reproduce meaningful fractal structure and instead generates amorphous coloured blobs, usually reddish because two of the three palettes are dominated by warm tones. On the Hopalong dataset, the global structure is partially preserved although not faithfully reproduced. On the Dream-only dataset, the GAN again fails to capture the true curves of the attractor, occasionally producing shapes that vaguely resemble fragments of the original geometry.

The behaviour of the GAN also reflects the structural and colour variability of the datasets. On the multi-attractor dataset, where both geometry and palette distribution vary widely, the model fails to reproduce meaningful fractal structure and instead generates amorphous colour blobs, often reddish because two of the three palettes contain warm tones. On the Hopalong dataset, where the global structure is stable despite the mixed palettes, the GAN preserves part of the overall layout, although without reproducing the characteristic fine curves of the attractor. On the Dream-only dataset, which features high structural variability under a

fixed palette, the GAN again struggles to capture the underlying geometry and produces diffuse shapes that only occasionally resemble fragments of the true Dream patterns.

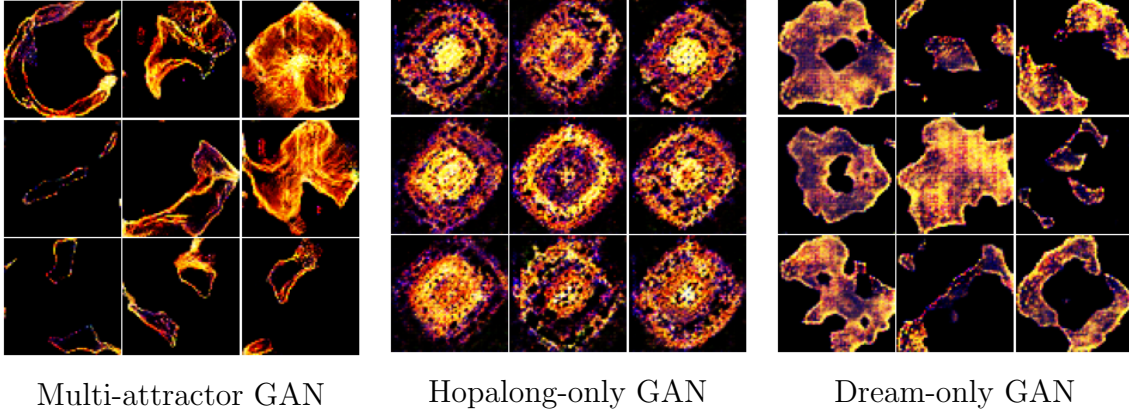


Figure 4: GAN-generated samples for the three datasets.

Latent-space interpolations Figure 5 shows three latent interpolations, all computed on the Hopalong dataset. The first interpolates between two real images encoded by the VAE. The second interpolates between two random latent vectors decoded by the VAE. The third interpolates between two random latent vectors passed through the GAN generator. These visualisations allow us to qualitatively assess the smoothness and consistency of the learned latent spaces.

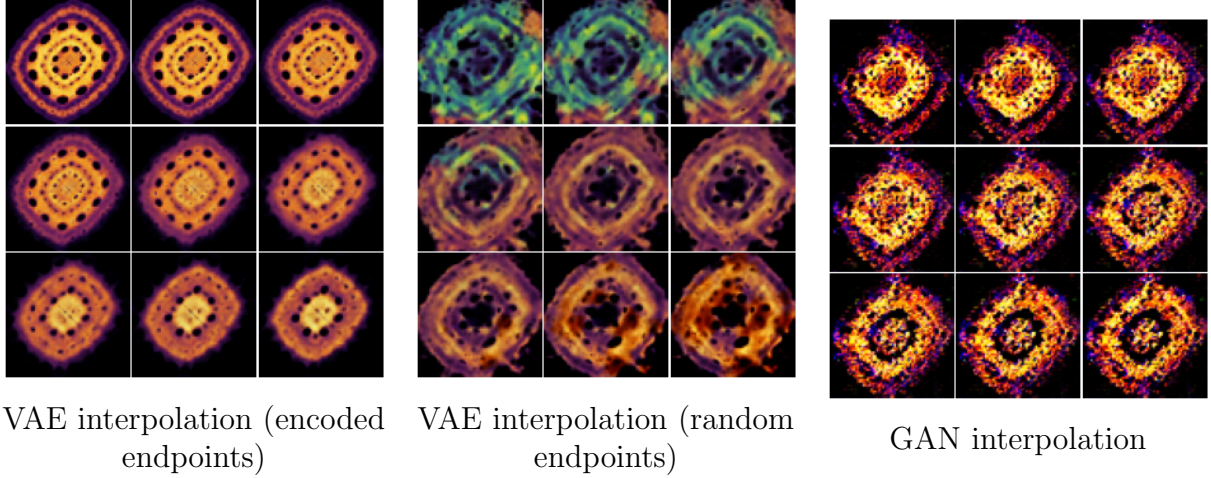


Figure 5: Latent-space interpolations for VAE and GAN on Hopalong-only dataset.

Discussion

Across all datasets, the CAE reconstructs the global structure but loses fine detail. As a purely reconstructive model, it has no mechanism to learn or reproduce the statistical distribution of the data, and therefore cannot be used to generate novel samples. The VAE consistently produces smooth and dark images, with limited ability to represent high-variability datasets and with frequent colour mixing when multiple palettes are present. The GAN generates sharper outputs and performs best on the Hopalong dataset, but fails to reproduce the geometry of both the multi-attractor and Dream datasets, where it collapses to amorphous colour patterns. These results highlight the limitations of simple generative models when applied to datasets with sparse bright regions, heterogeneous colour distributions, and complex geometric structure. Overall, the experiments confirm that model performance is strongly determined by the balance between structural variability and palette consistency, with stable single-attractor datasets being the most favourable scenario for both the VAE and the GAN.

2 Sequence modelling with recurrent neural networks

This task introduces an encoder–decoder recurrent network for simple arithmetic. The model learns to add or subtract two integers of up to two digits. Each input is a 5-character sequence (e.g., 89+20) and the model must output a 3-character result (e.g., 109). Outputs may include negatives (e.g., -48) or trailing spaces for numbers under 100 (e.g., 15), covering a value range of [-99, 198].

The sequences are build from 13 unique characters: the digits 0–9, the operators + and -, and the space character. These can be represented as raw strings, one-hot vectors, or images. The image form uses stacked MNIST digit images to represent each character (Appendix A). Since string and one-hot formats are bijectively mapped, they are functionally equivalent and used interchangeably in this report. In the subtasks, these two fall under Text, while the MNIST-based format falls under Image.

Using these representations, this report examines how well an encoder–decoder recurrent network performs simple arithmetic when using different input–output representations: text-to-text, image-to-text, and text-to-image.

Experimental Setup The encoder–decoder architecture is well-suited for sequence-to-sequence tasks like arithmetic. The encoder compresses the input into a fixed-size state vector, which the decoder expands into the output sequence. In our implementation, both components are LSTM layers.

Each model is evaluated using 50/50, 25/75, and 10/90 train/test splits; the Text-to-Image model also uses a 90/10 split because its task is more demanding. Large test sets are used to prioritize generalization over memorization. All models train for up to 150 epochs with batch size 32, using AdamW with a 0.01 learning rate. The learning rate is halved after 5 epochs without validation improvement, and training stops early after 20 such epochs. Early stopping is based on Cross-Entropy loss, except for the Text-to-Image model, which uses Mean Absolute Error.

2.1 Text-to-Text

The Text-to-Text RNN is our **base model**, handling the simplest task that all later models extend. It uses two stacked LSTMs with 256 hidden units: the first encodes the 5-character input into a 256-dimensional context vector, and the second decodes it into a 3-step (3, 256) sequence. A final fully connected layer with softmax maps each step to a (3, 13) one-hot text representation.

Results Across the three train/test splits, we obtained the results shown in Table 1. Performance clearly declines as the training set becomes smaller. The 10/90 split performs notably worse, which is expected given that 10% training data is insufficient for the model to learn the arithmetic patterns fully.

Train/test split	Test accuracy	Epochs
50/50	99.59%	150
25/75	97.99%	150
10/90	77.03%	106 (early stop)

Table 1: Final performance results of all three Text-to-Text networks

To understand where errors occur, we examine Figure 7. The top row shows histograms of the residuals ($y_{pred} - y_{true}$), and the bottom row shows confusion matrices containing only misclassified characters. The histograms are normally distributed and reveal that most errors are small and roughly centred around zero. The confusion matrix values are mostly uniformly distributed close to the diagonal. There are very few confusions where the special characters (+, -, ' ') are confused with a number. These types of errors would indicate the decoder's inability to decode the context vector to its corresponding sequence.

One notable exception is the frequent confusion between 9 and 1, which appears even in the stronger models. This is unexpected, as one would typically expect 9 to be mistaken for 0 instead (a pattern seen only in the weaker 10/90 model). The 9→1 confusion suggests difficulty with missclassified boundary cases involving carry-over logic. For instance, when the encoder inaccurately classifies the 99 as 100, the decoder also fails in the carry-over logic and represents the 100 as 10. This makes it seem like the encoder and decoder fail.

These results indicate that most of the errors are numerical mistakes caused by the encoder's inability to fully capture the arithmetic operation into the context vector. Additionally, the decoder struggles with handling the edge cases where it needs to apply carry-over logic of missclassified inputs.

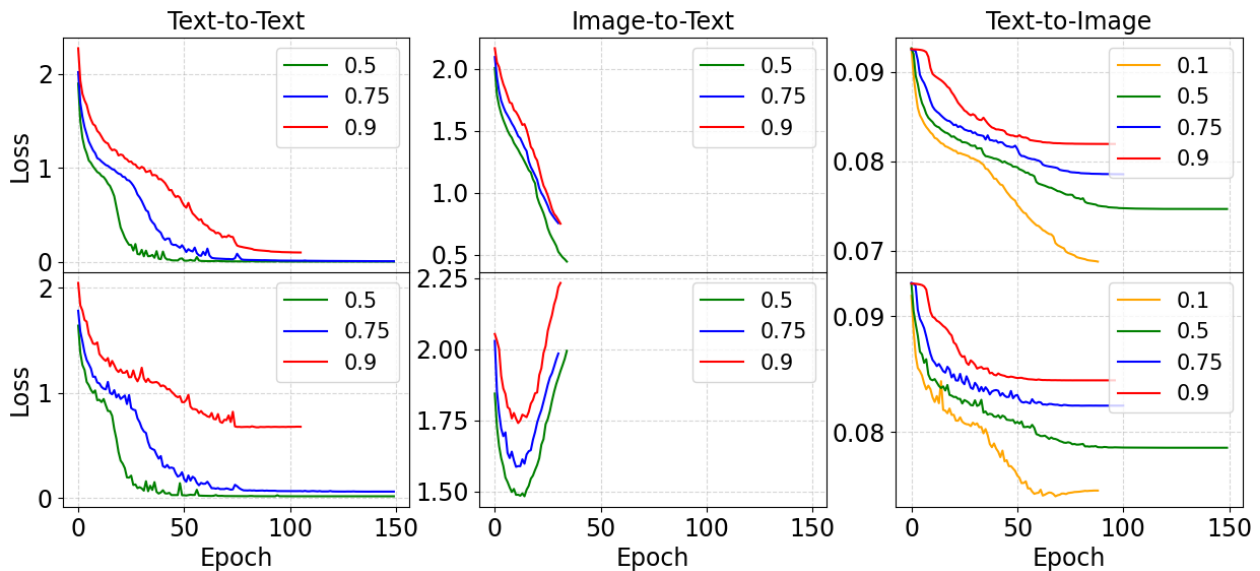


Figure 6: Loss vs. epochs for the three sequence-to-sequence models. Columns: Text-to-Text, Image-to-Text, Text-to-Image; top = train, bottom = validation; legends show test fraction.

2.2 Image-to-Text

The Image-to-Text model is similar to Text-to-Text, but takes flattened images as input and uses a 512-unit encoder for greater capacity. As Table 2 and the middle column of Figure 6 show, it struggles to generalize and begins overfitting after 10–15 epochs.

Poor performance may stem from LSTMs being ill-suited for raw images or from insufficient network capacity. Adding another LSTM layer could test the latter, as discussed in Section 2.4.

The large performance gap between the Text-to-Text and Image-to-Text models is unsurprising given the difference in problem complexity. Text-to-Text uses low-dimensional, well-separated one-hot inputs, so the model only needs to learn a straightforward deterministic

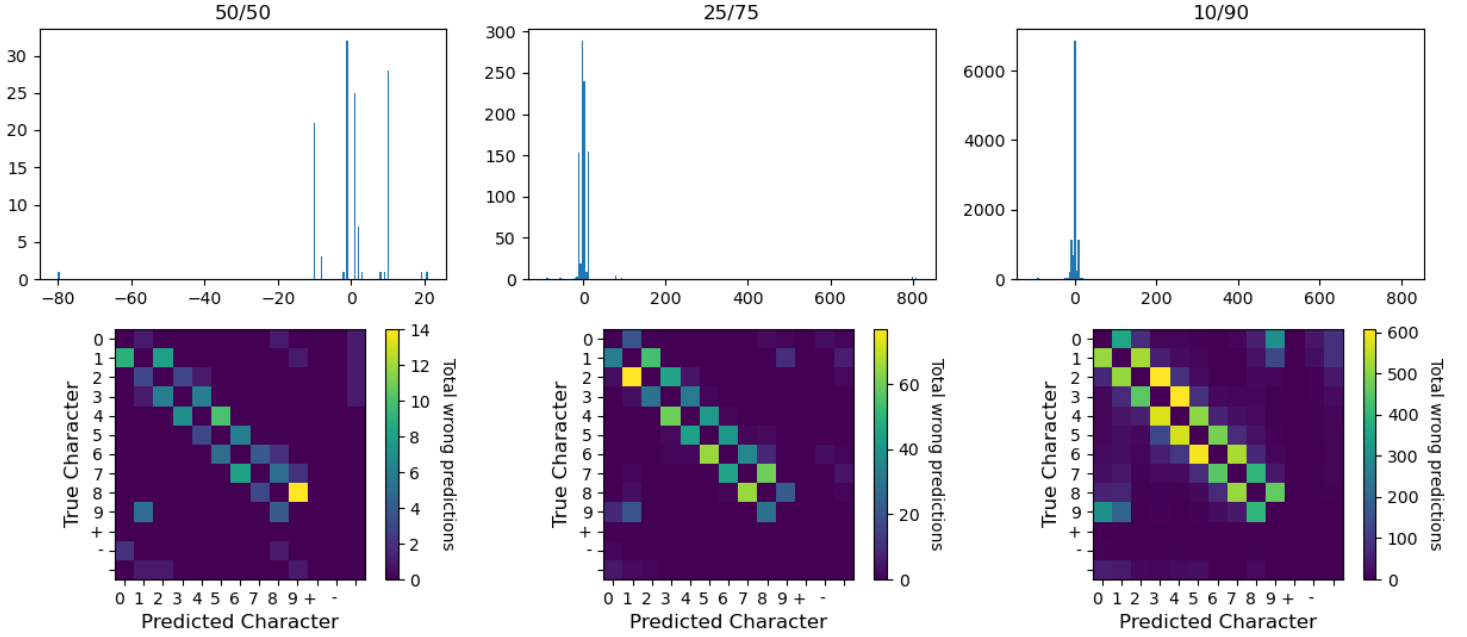


Figure 7: Error overview for Text-to-Text models. Histograms of $y_{pred} - y_{true}$ and confusion matrices on character level (errors only) per traintest split.

Train/test split	Test accuracy	Epochs
50/50	48.91%	35 (early stop)
25/75	44.01%	31 (early stop)
10/90	39.17%	32 (early stop)

Table 2: Final performance results of all three Image-to-Text networks

mapping. Image-to-Text, while also deterministic, has much higher-dimensional inputs that cannot be directly mapped to one-hot vectors, requiring the network to also learn the representation mapping itself. Combined with the fact that LSTMs are not well-suited for image processing, this makes the Image-to-Text task considerably harder.

2.3 Text-to-Image

Evaluating the Text-to-Image model is less straightforward than in earlier experiments. MAE alone cannot ensure that the generated image actually depicts the correct output. To address this, we introduce an independently trained CNN that classifies the output images. If this supervisor CNN correctly identifies a generated image, we assume the model produced the intended character. The CNN is trained on individual character images of the output Image sequences and their labels.

Supervisor CNN The CNN is composed of four convolutional blocks with increasing channel depth (16, 32, 64, and 128 filters). Each block consists of two convolution layers with GELU or ReLU activation, L2 regularization, and batch normalization, followed by max pooling. After the final block, global average pooling is applied. The resulting features are passed through a 64-unit fully connected layer with ReLU activation and outputs a vector of size 13 with an softmax activation.

The CNN was trained for 25 epochs using AdamW with a learning rate of 0.01, reduced by

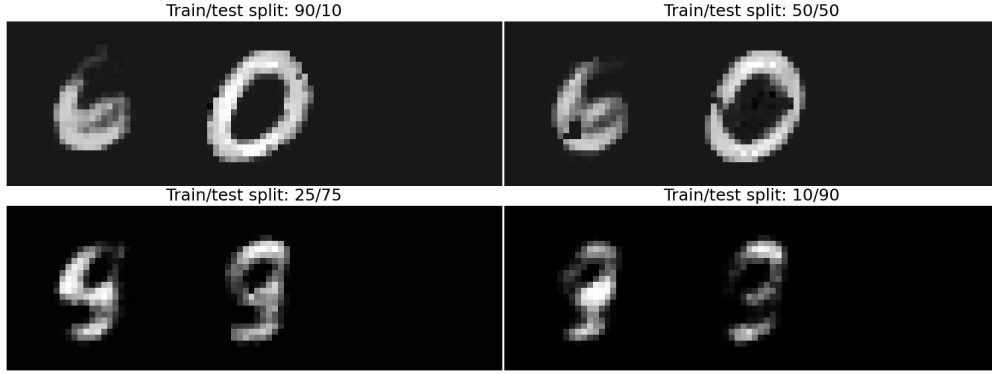


Figure 8: Example of the Text-to-Image networks output for all different train/test splits. The images are supposed to show the number 60

a factor of 10 after two plateauing validation epochs. Using cross-entropy loss, it reached 99.63% accuracy on the 10% validation set and 99.92% on the training set, indicating it is well suited to act as a supervisor for the Text-to-Image model.

Text-to-Image model This model builds on the Text-to-Text architecture, keeping the encoder unchanged. The decoder drops the fully connected layer and adds two LSTMs: the first with 256 hidden units and the second with 784, all returning sequences of length 3. The output is then reshaped to (3, 28, 28) to form the generated image sequence.

Since creating images is a more challenging task, we’ve additionally tested the model’s performance on a 90/10% train/test split. When observing the convergence plots (Figure 6), it becomes apparent that the network exhibits early stopping behavior on all train/test splits, with the exception of the 50%/50% split. The minimum loss also decreases as the training size increases, which is to be expected.

Visual inspection of the generated images (Figure 8) shows that the 90/10 and 50/50 models produce recognizable outputs. Further evaluation used a supervisor CNN. The generated sequences were split into individual images with labels and classified, yielding the results in Table 3.

Train/test split	Accuracy	Accuracy (no blanks)
90/10	96.95%	96.19%
50/50	78.92%	73.67%
25/75	61.76%	52.24%
10/90	53.09%	41.42%

Table 3: Accuracy of the supervisor CNN on images generated using various train/test splits and with or without blank images.

We can conclude that the 90/10 network, in particular, produces realistic images. Overall, the CNN classification aligns with the visual quality of the generated outputs. However, the lower performing splits (25/75 and 10/90) show higher than expected accuracies. This is likely because many generated images are simple black images that even the lower performing networks can generate consistently. Excluding these blanks reduces their accuracy by $\sim 10\%$, indicating that the apparent performance of the weaker networks is somewhat misleading.

2.4 Extending Encoder

This section examines how adding an LSTM layer to the encoder affects model accuracy. The experiments use the same setups, allowing a direct comparison with the previous networks.

Task		Text-to-Text	Image-to-Text	Text-to-Image
Old	50/50	99.59%	48.91%	78.92%
	25/75	97.99%	44.58%	61.76%
	10/90	77.03%	40.32%	53.09%
New	50/50	99.88%	68.93%	87.12%
	25/75	98.82%	51.06%	66.59%
	10/90	66.35%	42.71%	54.84%

Table 4: Accuracy change after adding an extra encoder LSTM layer (“Old” = 1 layer, “New” = 2 layers).

It appears that only the Text-to-Text task does not benefit significantly from adding an extra LSTM layer, as there is very little performance gain for the 50/50% and 25/75% splits. On average, it even performs worse for the 10/90% train/test split, where accuracy decreases by approximately 10%. This could be due to the increase in model capacity also increasing the risk of overfitting. For the other tasks, the networks do benefit from the additional LSTM layer. However, as the fraction of the training set decreases, the benefit of the extra layer diminishes. This is likely because the reduced training data is insufficient for the model to fully learn the input space.

Adding an extra layer increases the model’s capacity, which can reduce bias and improve performance. However, if the dataset is too small, the additional capacity may lead to overfitting, which is reflected in the lower accuracies for smaller training sets.

2.5 Discussion

In the sequence modeling tasks, the Text-to-Text model achieved high accuracy, with errors primarily stemming from numerical mistakes. The Image-to-Text model performed significantly worse, suffering from early overfitting as the LSTM architecture struggled to map high-dimensional image inputs to text representations. A potential solution is to first transform the input image into a Text format before the RNN to perform the arithmetic operations. This intermediate representation can then be utilized by incorporating it through a skip connection directly to the output during training. This approach separates the perception step from the reasoning step, which can simplify learning and improve interpretability. The Text-to-Image model, evaluated via a supervisor CNN, showed that larger training splits produced recognizable outputs, whereas smaller splits often resulted in blank images that misleadingly inflated accuracy scores. Adding an extra LSTM layer to the encoder improved performance for the image-based tasks but caused slight overfitting in the Text-to-Text model when training data was scarce.

3 Conclusion

Overall, the study confirms that while recurrent networks are well-suited for deterministic text-based arithmetic, their effectiveness diminishes significantly when processing image modalities without sufficient data or model capacity. Similarly, the generative experiments illustrate that standard VAE and GAN architectures require more consistent data structures to generate high-fidelity fractal images.

Contribution

Name	Task	Report or Code
Mario Magalini & Coen Büller	Introduction	report
Mario Magalini	Task 1	report & code
Coen Büller	Task 2	report & code
Mario Magalini & Coen Büller	Conclusion	report

Appendix

A Example input/output sample



Figure 9: This figure shows an example input and output sample in the image format. The left image is the input, while the right is the output. The input sample in string format would be "3-23", while the output would be "-20". The one-hot encoding of these sequences is shown in the paragraph below

To represent the input and output sequences of 9 you'd need an array of length 13 for each character in the sequence. Table 5 demonstrates how the first character of the input ('3') is represented using a one-hot encoding.

0	1	2	3	4	5	6	7	8	9	+	-	' '
0	0	0	1	0	0	0	0	0	0	0	0	0

Table 5: This table shows how a 3 is represented using a one-hot encoding. The bottom row is the actual encoding, and the upper row shows what the value of the encoding at that index represents. If there is a 1 at a given in the encoding, then the one-hot encoding represents the character in the upper row that is at the same index as the 1 in the encoding. This means that in your one-hot encoding, you can have at max one 1. In the example you can clearly see that in the bottom row at index 4 there is a one, and in the upper row at index 4 it has the value three, which means that the entire encoding at the bottom row represents a 3.

B Architecture provided from the Jupiter notebook

References

- [1] Lgoareguer (2019). Fractal dream dataset.
- [2] Magalini, M. (2025). Custom fractals dataset. <https://huggingface.co/datasets/FisicoCurioso/Fractals>.

Model	Architecture	Main Hyperparameters	Activations	Loss Functions	Optimizer
CAE	Encoder: Conv2D (128 filters, k=3, s=2, ReLU) $\times$ 5 $\rightarrow$ Flatten $\rightarrow$ Dense(512). Decoder: Dense($4 \times 4 \times 64$) $\rightarrow$ Reshape ($4, 4, 64$) $\rightarrow$ Conv2DTranspose (128 filters, s=2, ReLU) $\times$ 4 $\rightarrow$ Conv2D(3, Sigmoid).	latent_dim = 512; filters = 128; batch_size = 64; epochs = 50	ReLU + Sigmoid	Binary cross-entropy	Adam
VAE	Encoder: Conv2D (64 filters, k=3, s=2, ReLU) $\times$ 5 $\rightarrow$ Flatten $\rightarrow$ Dense($2 \times \text{latent_dim}$) $\rightarrow z_{\text{mean}} + z_{\text{logvar}} \rightarrow$ Sampling layer. Decoder: Dense($4 \times 4 \times 64$) $\rightarrow$ Reshape $\rightarrow$ Conv2DTranspose (64 filters, s=2) $\times$ 4 $\rightarrow$ Conv2D(3, Sigmoid).	latent_dim = 64; filters = 64; batch_size = 8; epochs = 20	ReLU + Sigmoid	Binary cross-entropy + KL divergence	Adam (lr = 10^{-3})
GAN – Generator	Dense($4 \times 4 \times 64$) $\rightarrow$ Reshape $\rightarrow$ Conv2DTranspose (128 filters, s=2) $\times$ 4 $\rightarrow$ Conv2D(3, Tanh).	latent_dim = 256; filters = 128	ReLU + Tanh	—	Adam (lr = 0.0002, $\beta_1=0.5$)
GAN – Discriminator	Conv2D (128 filters, k=3, s=2, ReLU) $\times$ 5 $\rightarrow$ Flatten $\rightarrow$ Dense(1, Sigmoid).	batch_size = 64; epochs $\approx$ 20	ReLU + Sigmoid	Binary cross-entropy	Adam (lr = 0.0002, $\beta_1=0.5$)
GAN – Training Loop	D: trained with real/fake half-batches. G: trained through GAN with target label 1.	batches_per_epoch = $N/\text{batch_size}$	—	Binary cross-entropy (both D and G)	Adam (same hyperparameters)

Table 6: Summary of architectures and hyperparameters used in the provided CAE, VAE and GAN implementations.